

```
# %%capture
!pip install -q accelerate==0.31.0 peft==0.11.1 bitsandbytes==0.43.1 transformers==4.41.2 trl==0.9.4 sentencepiece==0.2.0 triton==3.1.0
```

Supervised Fine-Tuning (SFT)

Data Preprocessing

```
from transformers import AutoTokenizer
from datasets import load_dataset

# Load a tokenizer to use its chat template
template_tokenizer = AutoTokenizer.from_pretrained("TinyLlama/TinyLlama-1.1B-Chat-v1.0")

def format_prompt(example):
    """Format the prompt to using the <|user|> template TinyLlama is using"""

    # Format answers
    chat = example["messages"]
    prompt = template_tokenizer.apply_chat_template(chat, tokenize=False)

    return {"text": prompt}

# Load and format the data using the template TinyLlama is using
dataset = (
    load_dataset("HuggingFaceH4/ultrachat_200k", split="test_sft")
    .shuffle(seed=42)
    .select(range(3_000))
)
dataset = dataset.map(format_prompt)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:104: UserWarning:
Error while fetching `HF_TOKEN` secret value from your vault: 'Requesting secret HF_TOKEN timed out. Secrets can only be fetched when ru
You are not authenticated with the Hugging Face Hub in this notebook.
If the error persists, please let us know by opening an issue on GitHub (https://github.com/huggingface/huggingface\_hub/issues/new).
warnings.warn(
tokenizer_config.json: 0.00B [00:00, ?B/s]
tokenizer.model: 0% | 0.00/500k [00:00<?, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0% | 0.00/551 [00:00<?, ?B/s]
README.md: 0.00B [00:00, ?B/s]
data/train_sft-00000-of-00003-a3ecf92756(...): 0% | 0.00/244M [00:00<?, ?B/s]
data/train_sft-00001-of-00003-0a1804bcb6(...): 0% | 0.00/244M [00:00<?, ?B/s]
data/train_sft-00002-of-00003-ee46ed25cf(...): 0% | 0.00/244M [00:00<?, ?B/s]
data/test_sft-00000-of-00001-f7dfac4afe5(...): 0% | 0.00/81.2M [00:00<?, ?B/s]
data/train_gen-00000-of-00003-a6c9fb894b(...): 0% | 0.00/244M [00:00<?, ?B/s]
data/train_gen-00001-of-00003-d6a0402e41(...): 0% | 0.00/243M [00:00<?, ?B/s]
data/train_gen-00002-of-00003-c0db75b92a(...): 0% | 0.00/243M [00:00<?, ?B/s]
data/test_gen-00000-of-00001-3d4cd830914(...): 0% | 0.00/80.4M [00:00<?, ?B/s]
Generating train_sft split: 0% | 0/207865 [00:00<?, ? examples/s]
Generating test_sft split: 0% | 0/23110 [00:00<?, ? examples/s]
Generating train_gen split: 0% | 0/256032 [00:00<?, ? examples/s]
Generating test_gen split: 0% | 0/28304 [00:00<?, ? examples/s]
Map: 0% | 0/3000 [00:00<?, ? examples/s]
```

```
# Example of formatted prompt
print(dataset["text"][2576])
```

```
<|user|>
Given the text: Knock, knock. Who's there? Hike.
Can you continue the joke based on the given text material "Knock, knock. Who's there? Hike"?</s>
<|assistant|>
Sure! Knock, knock. Who's there? Hike. Hike who? Hike up your pants, it's cold outside!</s>
<|user|>
Can you tell me another knock-knock joke based on the same text material "Knock, knock. Who's there? Hike"?</s>
<|assistant|>
Of course! Knock, knock. Who's there? Hike. Hike who? Hike your way over here and let's go for a walk!</s>
```

Models - Quantization

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
```

```

model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"

# 4-bit quantization configuration - Q in QLoRA
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, # Use 4-bit precision model loading
    bnb_4bit_quant_type="nf4", # Quantization type
    bnb_4bit_compute_dtype="float16", # Compute dtype
    bnb_4bit_use_double_quant=True, # Apply nested quantization
)

# Load the model to train on the GPU
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",

    # Leave this out for regular SFT
    quantization_config=bnb_config,
)

model.config.use_cache = False
model.config.pretraining_tp = 1

# Load LLaMA tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=False)
tokenizer.pad_token = "<PAD>"
tokenizer.padding_side = "left"

```

```

config.json: 0%|          | 0.00/560 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/4.40G [00:00<?, ?B/s]
generation_config.json: 0%|          | 0.00/129 [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/776 [00:00<?, ?B/s]
tokenizer.model: 0%|          | 0.00/500k [00:00<?, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|          | 0.00/414 [00:00<?, ?B/s]

```

Configuration

LoRA Configuration

```

from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model

# Prepare LoRA Configuration
peft_config = LoraConfig(
    lora_alpha=32, # LoRA Scaling
    lora_dropout=0.1, # Dropout for LoRA Layers
    r=64, # Rank
    bias="none",
    task_type="CAUSAL_LM",
    target_modules= # Layers to target
    ['k_proj', 'gate_proj', 'v_proj', 'up_proj', 'q_proj', 'o_proj', 'down_proj']
)

# prepare model for training
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, peft_config)

```

Training Configuration

```

from transformers import TrainingArguments

output_dir = "./results"

# Training arguments
training_arguments = TrainingArguments(
    output_dir=output_dir,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    optim="paged_adamw_32bit",
    learning_rate=2e-4,
    lr_scheduler_type="cosine",
    num_train_epochs=1,
    logging_steps=10,
    fp16=True,
    gradient_checkpointing=True
)

```

Training!

```

from trl import SFTTrainer

# Set supervised fine-tuning parameters
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    dataset_text_field="text",
    tokenizer=tokenizer,
    args=training_arguments,
    max_seq_length=512,

    # Leave this out for regular SFT
    peft_config=peft_config,
)

# Train model
trainer.train()

# Save QLoRA weights
trainer.model.save_pretrained("TinyLlama-1.1B-qlora")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_deprecation.py:100: FutureWarning: Deprecated argument(s) used in '__init__'
  warnings.warn(message, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/training_args.py:1965: FutureWarning: `--push_to_hub_token` is deprecated and will be removed in a future version. Please use `--push_to_hub_repo` instead.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/trl/trainer/sft_trainer.py:269: UserWarning: You passed a `max_seq_length` argument to the SFTTrainer, but it is deprecated. Please use `args.max_seq_length` instead.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/trl/trainer/sft_trainer.py:307: UserWarning: You passed a `dataset_text_field` argument to the SFTTrainer, but it is deprecated. Please use `args.dataset_text_field` instead.
  warnings.warn(
Map:   0%|          | 0/3000 [00:00<?, ? examples/s]
/usr/local/lib/python3.12/dist-packages/trl/trainer/sft_trainer.py:397: UserWarning: You passed a tokenizer with `padding_side` not equal to `left`, while the model has `padding_side` set to `left`. This might lead to unexpected behavior.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/accelerate/accelerator.py:477: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated in favor of `torch.cuda.amp.GradScaler`.
  self.scaler = torch.cuda.amp.GradScaler(**kwargs)
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not intended, please specify a different `run_name`.
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence 'W/'
  | l_| | '_ W/ _` / _` | _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice:wandb: You chose 'Create a W&B account'
wandb: Create an account here: https://wandb.ai/authorize?signup=true&ref=models
wandb: Paste an API key from your profile and hit enter:

```

Merge Adapter

```

from peft import AutoPeftModelForCausalLM

model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora",
    low_cpu_mem_usage=True,
    device_map="auto",
)

# Merge LoRA and base model
merged_model = model.merge_and_unload()

```

▼ Inference

```

from transformers import pipeline

# Use our predefined prompt template
prompt = """<user|>
Tell me something about Large Language Models.</s>
<|assistant|>
"""

# Run our instruction-tuned model
pipe = pipeline(task="text-generation", model=merged_model, tokenizer=tokenizer)
print(pipe(prompt)[0]["generated_text"])

```

```

<|user|>
Tell me something about Large Language Models.</s>
<|assistant|>
Large Language Models (LLMs) are a type of artificial intelligence (AI) that can generate human-like language. They are trained on large
LLMs are used in a variety of applications, including natural language processing (NLP), machine translation, and chatbots. They can be
One of the most significant applications of LLMs is in the field of natural language generation (NLG). LLMs can be used to generate text
LLMs have the potential to revolutionize the way we communicate and interact with each other. They can help us create more engaging and

```

▼ Preference Tuning (PPO/DPO)

▼ Data Preprocessing

```

from datasets import load_dataset

def format_prompt(example):
    """Format the prompt to using the <|user|> template TinyLlama is using"""

    # Format answers
    system = "<|system|>\n" + example['system'] + "</s>\n"
    prompt = "<|user|>\n" + example['input'] + "</s>\n<|assistant|>\n"
    chosen = example['chosen'] + "</s>\n"
    rejected = example['rejected'] + "</s>\n"

    return {
        "prompt": system + prompt,
        "chosen": chosen,
        "rejected": rejected,
    }

# Apply formatting to the dataset and select relatively short answers
dpo_dataset = load_dataset("argilla/distilabel-intel-orca-dpo-pairs", split="train")
dpo_dataset = dpo_dataset.filter(
    lambda r:
        r["status"] != "tie" and
        r["chosen_score"] >= 8 and
        not r["in_gsm8k_train"]
)
dpo_dataset = dpo_dataset.map(format_prompt, remove_columns=dpo_dataset.column_names)
dpo_dataset

```

```

Downloading readme: 0%|          | 0.00/10.1k [00:00<?, ?B/s]
Downloading data: 0%|          | 0.00/79.2M [00:00<?, ?B/s]
Generating train split: 0%|        | 0/12859 [00:00<?, ? examples/s]
Filter: 0%|          | 0/12859 [00:00<?, ? examples/s]
Map: 0%|            | 0/5922 [00:00<?, ? examples/s]
Dataset({
  features: ['chosen', 'rejected', 'prompt'],
  num_rows: 5922
})

```

Models - Quantization

```

from peft import AutoPeftModelForCausalLM
from transformers import BitsAndBytesConfig, AutoTokenizer

# 4-bit quantization configuration - Q in QLoRA
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, # Use 4-bit precision model loading
    bnb_4bit_quant_type="nf4", # Quantization type
    bnb_4bit_compute_dtype="float16", # Compute dtype
    bnb_4bit_use_double_quant=True, # Apply nested quantization
)

# Merge LoRA and base model
model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora",
    low_cpu_mem_usage=True,
    device_map="auto",
    quantization_config=bnb_config,
)
merged_model = model.merge_and_unload()

# Load LLaMA tokenizer
model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=False)
tokenizer.pad_token = "<PAD>"
tokenizer.padding_side = "left"

```

```

/usr/local/lib/python3.10/dist-packages/peft/tuners/lora/bnb.py:325: UserWarning: Merge lora module to 4-bit linear may get different ge
warnings.warn(

```

Configuration

```

from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model

# Prepare LoRA Configuration
peft_config = LoraConfig(
    lora_alpha=32, # LoRA Scaling
    lora_dropout=0.1, # Dropout for LoRA Layers
    r=64, # Rank
    bias="none",
    task_type="CAUSAL_LM",
    target_modules= # Layers to target
    ['k_proj', 'gate_proj', 'v_proj', 'up_proj', 'q_proj', 'o_proj', 'down_proj']
)

# prepare model for training
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, peft_config)

```

```

from trl import DPOConfig

output_dir = "./results"

# Training arguments
training_arguments = DPOConfig(
    output_dir=output_dir,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    optim="paged_adamw_32bit",
    learning_rate=1e-5,
    lr_scheduler_type="cosine",
    max_steps=200,

```

```

        logging_steps=10,
        fp16=True,
        gradient_checkpointing=True,
        warmup_ratio=0.1
    )

```

```

from trl import DPOTrainer

# Create DPO trainer
dpo_trainer = DPOTrainer(
    model,
    args=training_arguments,
    train_dataset=dpo_dataset,
    tokenizer=tokenizer,
    peft_config=peft_config,
    beta=0.1,
    max_prompt_length=512,
    max_length=512,
)

# Fine-tune model with DPO
dpo_trainer.train()

# Save adapter
dpo_trainer.model.save_pretrained("TinyLlama-1.1B-dpo-qlora")

```

```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_deprecation.py:100: FutureWarning: Deprecated argument(s) used in '__init__'
Deprecated positional argument(s) used in DPOTrainer, please use the DPOConfig to set these arguments instead.
  warnings.warn(message, FutureWarning)
/usr/local/lib/python3.10/dist-packages/peft/tuners/lora/bnb.py:325: UserWarning: Merge lora module to 4-bit linear may get different ge
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/trl/trainer/dpo_trainer.py:358: UserWarning: You passed `max_length` to the DPOTrainer, the valu
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/trl/trainer/dpo_trainer.py:371: UserWarning: You passed `max_prompt_length` to the DPOTrainer, t
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/trl/trainer/dpo_trainer.py:411: UserWarning: When using DPODataCollatorWithPadding, you should s
  warnings.warn(
Map:   0%|          | 0/5922 [00:00<?, ? examples/s]
max_steps is given, it will override any value given in num_train_epochs
/usr/local/lib/python3.10/dist-packages/torch/utils/checkpoint.py:464: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/torch/utils/checkpoint.py:91: UserWarning: None of the inputs have requires_grad=True. Gradients
  warnings.warn(
Could not estimate the number of tokens of the input, floating-point operations will not be computed
[200/200 12:52, Epoch 0/1]

```

Step	Training Loss
------	---------------

10	0.692400
20	0.678200
30	0.646000
40	0.606300
50	0.595600
60	0.616800
70	0.593700
80	0.531900
90	0.559200
100	0.639000
110	0.496500
120	0.586000
130	0.630000
140	0.590100
150	0.577500
160	0.591000
170	0.606900
180	0.627800
190	0.668600

```

from peft import PeftModel

```

```
# Merge LoRA and base model
model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora",
    low_cpu_mem_usage=True,
    device_map="auto",
)
sft_model = model.merge_and_unload()

# Merge DPO LoRA and SFT model
dpo_model = PeftModel.from_pretrained(
    sft_model,
    "TinyLlama-1.1B-dpo-qlora",
    device_map="auto",
)
dpo_model = dpo_model.merge_and_unload()
```

```
from transformers import pipeline

# Use our predefined prompt template
prompt = """<|user|>
Tell me something about Large Language Models.</s>
<|assistant|>
"""

# Run our instruction-tuned model
pipe = pipeline(task="text-generation", model=dpo_model, tokenizer=tokenizer)
print(pipe(prompt)[0]["generated_text"])
```

```
<|user|>
Tell me something about Large Language Models.</s>
<|assistant|>
Large Language Models (LLMs) are a type of artificial intelligence (AI) that can generate human-like language. They are trained on large
LLMs are used in a variety of applications, including natural language processing (NLP), machine translation, and chatbots. They can be
One of the most significant applications of LLMs is in the field of natural language generation (NLG). LLMs can be used to generate text
LLMs have the potential to revolutionize the way we communicate and interact with each other. They can help us create more engaging and
```